DC MOTOR PI TUNING USING REINFORCEMENT LEARNING

1.0 Learning Outcomes.....1
2.0 Hardware Set-Up.....1
3.0 Practical Investigation via Simulink.....1
4.0 Concluding Remarks.....5
Recommended Citation.....6

1.0 Learning Outcomes

Upon completing this practical exercise, students should be able to demonstrate the Learning Outcomes presented in Table 1. At the end of the exercise, revisit each Learning Outcome and confirm that it has been achieved.

Table 1: *Learning Outcomes for the Practical Exercise*

ID	Learning Outcome	Achieved
1	Build a Simulink training environment in which a reinforcement learning (RL) agent tunes the gains of a PI speed controller, and identify its observations, actions and reward.	☐
2	Train a TD3 agent on randomised virtual DC motors, and interpret the training progress.	☐
3	Evaluate the trained RL tuner against the engineering requirements when the motor changes, and identify the evidence needed before it could be trusted on hardware.	☐

2.0 Hardware Set-Up

This practical exercise is carried out in simulation, using a virtual model of the ACE-Lab DC motor, so no hardware is needed. It builds on the practical exercises, DC Motor Speed Control Using a PI Controller, and DC Motor Self-Tuning PI Control using Recursive Least Squares, which introduce the PI speed loop, the engineering requirements and the software load. Keep the folder structure of the supplied files.

In addition to MATLAB and Simulink, the Reinforcement Learning Toolbox and the Deep Learning Toolbox are required. The Parallel Computing Toolbox is optional but strongly recommended: training takes about 25 minutes in parallel on a 16-core computer, and considerably longer on a computer with fewer cores.

3.0 Practical Investigation via Simulink

MATLAB and Simulink should be used to complete this practical exercise. Here, a TD3 agent learns how to choose the gains from experience, as described in the Reinforcement Learning for PI Controller Tuning Learning Guide. The agent supervises the PI controller: every 3 s it observes the performance over the last

reference step and chooses new values of K_p and K_i. The supplied files are listed in Table 2.

Table 2: *Files Used in this Practical Exercise (RL_PI_Tuner Folder)*

File	Purpose
`build_rl_env.m`	Builds the training environment, `rl_pi_tuner.slx`
`rl_block_code.m`	Code of the PI Controller, Performance Features and Action to Gains blocks
`randomMotor.m`	Chooses a random virtual motor and load change for each training episode
`train_rl_tuner.m`	Creates and trains the TD3 agent; saves `rl_pi_tuner_agent.mat`
`analyse_hw_run.m`	Checks a run against the engineering requirements (R1 to R5)
`export_rl_policy.m`, `build_rl_hw.m`	Export the trained policy as MATLAB code and build a hardware model (extension)

Build and Explore the Training Environment

1. Run `build_rl_env.m`, and open `rl_pi_tuner.slx`. Identify the Speed Reference, the PI Controller, the virtual motor and encoder, the Performance Features block, the RL Agent block and the Action to Gains block, as in Figure 1.
2. Double-click the Performance Features block, and identify the observations and the reward. The reward penalises the RMS speed error, the overshoot and the chatter (the mean change in PWM command per sample) over each 3 s interval:

$$R = -\left(\frac{e_{RMS}}{5}\right)^2 - \frac{OS}{2} - 0.15\,c \tag{1}$$

3. Double-click the Action to Gains block. The two actions, between −1 and 1, are mapped to K_p between 0.003 and 2 V/RPM and K_i between 0.05 and 8 V/(RPM s), on a logarithmic scale.

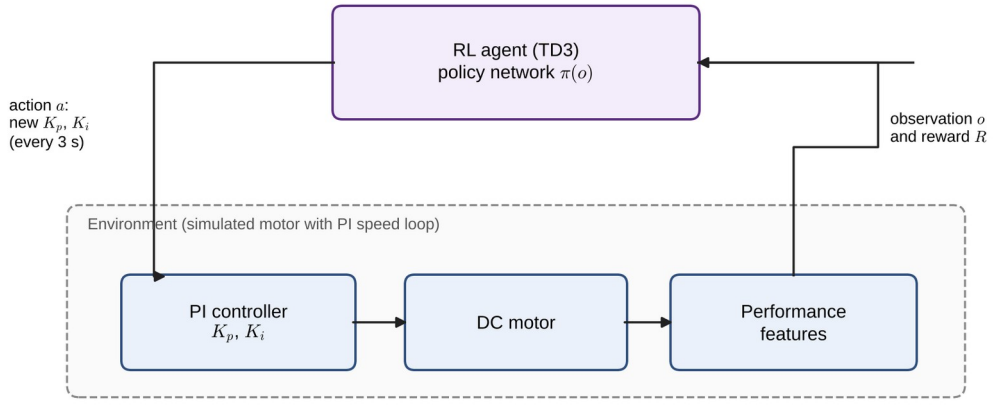


Figure 1: *The Training Environment: the RL Agent Chooses the PI Gains Every 3 s from Observations of the Speed Loop*

Choose the Training Motors

Open `randomMotor.m`. At the start of every training episode, it chooses a new virtual motor, with a random gain, time constant and dead-band, and a random load change: the gain of the motor is multiplied by a factor between 0.6 and 2.0, at

a random time between 20 and 70 s. The agent can only be expected to work well on motors similar to those it meets in training.

1. Note the ranges in the supplied file, which are centred on a hardware ACE-Lab motor (K = 16 RPM/V, τ = 0.054 s and a 0.2 V dead-band).
2. The agent is evaluated in this practical exercise on virtual motors with τ = 0.25 s and a 0.5 V dead-band, starting at K = 16 RPM/V (scenario A) or K = 10 RPM/V (scenario B). Edit the ranges in `randomMotor.m` so that they include these motors, and record your ranges in Table 3.

Table 3: *Ranges of the Training Motors*

Parameter	Supplied range (hardware motor)	Your range (test motors)
Gain, K (RPM/V)	16 to 26	
Time constant, τ (s)	0.03 to 0.12	
Dead-band, V_{dead} (V)	0 to 0.5	
Load factor, α	0.6 to 2.0	0.6 to 2.0
Time of the load change (s)	20 to 70	20 to 70

Train the Agent

1. Run `train_rl_tuner.m`. It creates a TD3 agent with a small actor network (10–16–16–2, 482 parameters) and two critics, and trains it for 1000 episodes of 90 s, each with 30 decisions. The Reinforcement Learning Training Monitor shows the reward of each episode and the average over the last 20 episodes.
2. While the agent trains, sketch the expected shape of the training curve. How should the average reward change as the exploration noise is reduced?
3. When training has finished, record the final average reward. In the example, the average reward over the last 20 episodes was −57.5.

Evaluate the RL Tuner

The trained agent is tested on a virtual motor whose gain changes at t = 40 s, which has the same effect as the software load in the practical exercise, DC Motor Self-Tuning PI Control using Recursive Least Squares: in scenario A, the motor starts at K = 16 RPM/V and its gain falls to 60%; in scenario B, it starts at K = 10 RPM/V and its gain doubles. The reference alternates between 20 and 40 RPM every 3 s for 80 s.

MATLAB Script: testing the trained agent on a virtual motor whose gain changes

```
S = load('rl_pi_tuner_agent.mat', 'agent');
assignin('base', 'agent', S.agent);          % used by the RL Agent block
mdl = 'rl_pi_tuner';  load_system(mdl);
scen = 'A';  K = 16;  alpha = 0.6;           % scenario B: K = 10, alpha = 2.0
motor = {'K_true', K; 'tau_true', 0.25; 'V_dead', 0.5; ...
         't_change', 40; 'K_factor', alpha};
in = Simulink.SimulationInput(mdl);
for j = 1:size(motor, 1)
    in = setVariable(in, motor{j,1}, motor{j,2}, 'Workspace', mdl);
end
in = setModelParameter(in, 'StopTime', '80');
out = sim(in);

res = analyse_hw_run(out, 'RL agent', scen, 40);   % R1 to R5
t = res.t;  e = res.r - res.y;
rmsB = sqrt(mean(e(t >= 20 & t < 40).^2));
```

```
rmsA = sqrt(mean(e(t >= 60 & t < 80).^2));
fprintf('RMS error %.2f RPM (20-40 s), %.2f RPM (60-80 s)\n', rmsB, rmsA);
subplot(211); plot(t, res.r, 'k--', t, res.y); grid on; ylabel('Speed (RPM)');
subplot(212); yyaxis left; plot(t, res.Kp); ylabel('K_p');
yyaxis right; plot(t, res.Ki); ylabel('K_i'); grid on; xlabel('Time (s)');
```

1. Run the script for scenario A, then change `scen`, `K` and `alpha` and run it for scenario B. `analyse_hw_run` checks every reference step from 10 to 37 s and from 50 to 80 s against the engineering requirements: rise time below 1 s (R1), steady-state error at most 0.1 RPM (R2), overshoot at most 5% (R3) and chatter at most 5 PWM counts per sample (R4). R5 is met if R1 to R4 are all met again from 50 s, after the change.

2. Record your results in Table 5, and compare them with the example results in Table 4.

   Example results are shown in Figure 2 and Table 4. The agent raised its gains when the motor weakened and lowered them when it became stronger, re-tuning within one or two decisions after each change, and met every requirement from 50 s in both scenarios, with a worst-case overshoot of 1.6%. Notice that the gains alternate at every reference step: the agent has learned to use different gains for upward and downward steps.

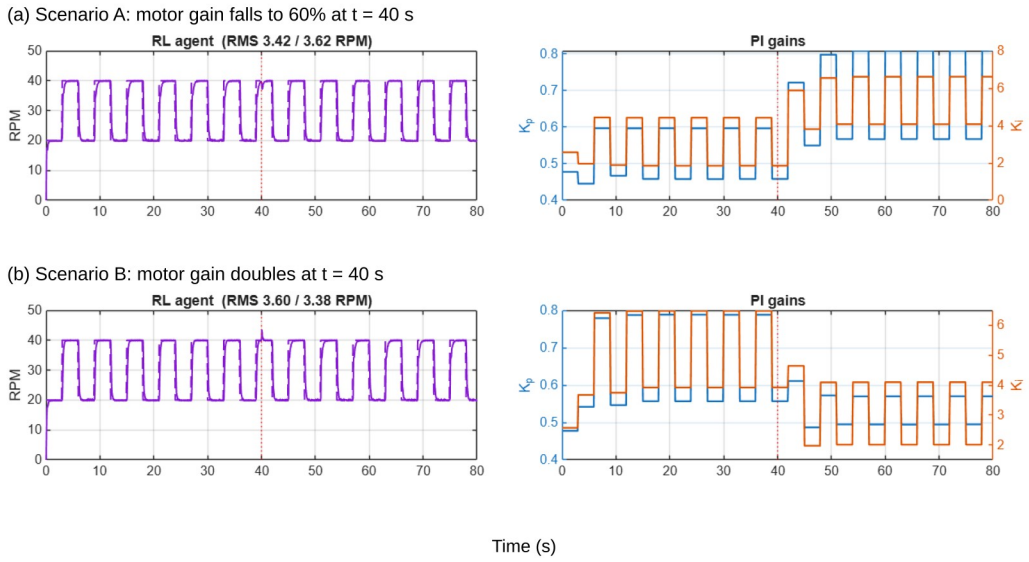


Figure 2: *Example Results: Speed (Left) and PI Gains Chosen by the RL Tuner (Right) for Scenario A (a) and Scenario B (b); the Gain of the Motor Changes at t = 40 s*

Table 4: *Example Results for the RL Tuner (Worst Case over the Steps in Each Window)*

Scenario	RMS error, 20–40 s (RPM)	RMS error, 60–80 s (RPM)	Worst overshoot, 10–40 s (%)	Worst overshoot, 50–80 s (%)	K_p, K_i at the end	R5 met
A	3.42	3.62	1.6	1.6	0.808, 6.616	Yes
B	3.60	3.38	17.6	1.6	0.570, 4.097	Yes

Table 5: *Your Results*

Scenario	RMS error, 20–40 s (RPM)	RMS error, 60–80 s (RPM)	Worst overshoot, 10–40 s (%)	Worst overshoot, 50–80 s (%)	K_p, K_i at the end	R5 met
A						
B						

Investigate the Reward and Generalisation

1. **Reward:** remove the chatter term from Equation (1) in `rl_block_code.m`, rebuild the environment and retrain for 300 episodes. Compare the gains chosen by the new agent, and the chatter, with those of the original agent. What does this tell you about how the agent uses the reward?
2. **Generalisation:** test the original agent on a motor outside its training range, for example τ = 0.6 s, or a load factor of 3, by changing `motor` in the script above. Does the agent still meet the requirements? Explain why, or why not.
3. **Extension (hardware):** `export_rl_policy.m` writes the trained actor as plain MATLAB code, and `build_rl_hw.m` builds `dc_motor_rl.slx`, in which the policy runs on the Arduino Mega 2560. An agent trained on the test motors is not suitable for the hardware motor, which is much faster: it must first be retrained with `randomMotor.m` centred on the identified hardware motor. If you test the hardware model, apply the same software load as in the other practical exercises, and keep a stop switch in reach.

Consider the following questions:

- In scenario B, the agent overshot by up to 17.6% before the change, but met every requirement after it. Why does a single reward not guarantee that every requirement is met?
- How would you explain to a client why the RL tuner chose K_p = 0.81 at the end of scenario A?
- What evidence would you need before trusting the RL tuner on a real product?

4.0 Concluding Remarks

This practical exercise trained a reinforcement learning agent to tune the gains of a PI speed controller. Using MATLAB, Simulink and the Reinforcement Learning Toolbox, a TD3 agent learned, over about 25 minutes of simulated experience on randomised virtual motors, to choose K_p and K_i every 3 s from observations of the tracking performance and the response of the motor, with a reward that penalised tracking error, overshoot and chatter. In simulation, the RL tuner re-tuned within one or two decisions after the motor changed, and then met every requirement in both scenarios.

This practical exercise also showed the limitations of learned control. The agent works only for motors similar to those it met in training, it optimises exactly what the reward measures, and its decisions cannot be read from the weights of its network. It must be retrained around the identified motor, and tested on the hardware across the full range of expected conditions, before it can be trusted.

Recommended Citation

If you have used this practical exercise in your teaching, learning or research, please cite it using the reference below, adapting the format as required by your preferred referencing style.

Pickering, J. E. (2026). DC Motor PI Tuning using Reinforcement Learning. ACE-Lab. Teaching Notes: ACE-Lab Practical Exercises, Version 1.0. Zenodo. https://doi.org/10.5281/zenodo.23671556